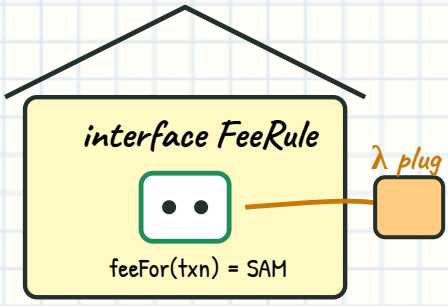
Functional interfaces · my notes L02

the contract behind every lambda: one abstract method = one socket

1 · SAM = Single Abstract Method



Counts: exactly 1 abstract method

Allowed, NOT counted:

- default methods [8]
- static methods [8]
- private methods [9]
- equals / hashCode / toString (Object's)

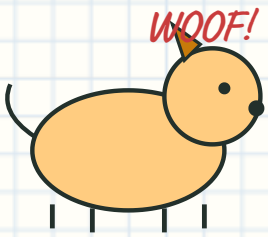
→ Comparator has compare + equals and is still functional.

2 · @FunctionalInterface = guard dog

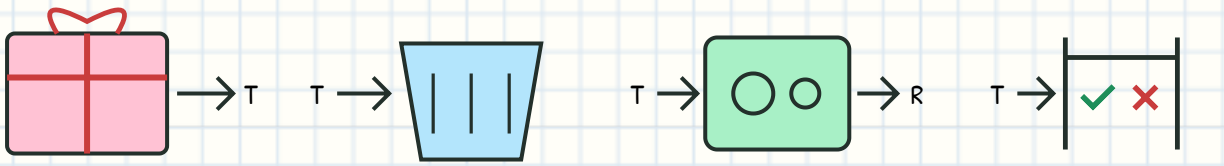
Optional. But if someone adds a 2nd abstract method, it barks **at the interface**, not in 50 other files:

error: Unexpected @FunctionalInterface annotation

Standard: always put it on my own SAMs.



3 · The four basic shapes java.util.function [8]



Supplier
() → T · get()

Consumer
T → void · accept()

Function
T → R · apply()

Predicate
T → boolean · test()

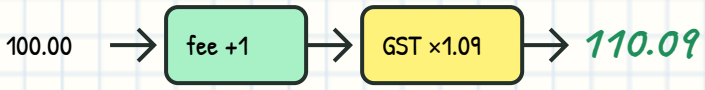
Bi… = 2 inputs
BiFunction (T,U) → R
BiPredicate, BiConsumer

…Operator = same type
UnaryOperator T → T
BinaryOperator (T,T) → T

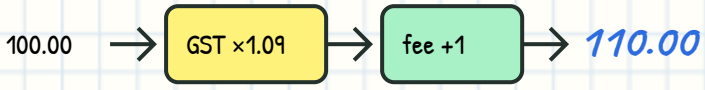
Primitives = no boxing
IntPredicate, ToIntFunction
IntUnaryOperator…

4 · Composition = snap the pieces together

andThen: this first



compose: argument first



Predicate logic (60 txns)

isDebit	45
isDebit.and(isLarge)	3
isDebit.or(isLarge)	53
isDebit.negate()	15
Predicate.not(isLarge) [11]	49

Before Java 8 I needed 5 classes (3 rules + AndCheck + NotCheck). Now:
`SUCCESSFUL.and(AT_LEAST_100).and(PAYPAL.negate())` → [P001, P005, P010].

5 · Boxing



Integer (boxed)

455

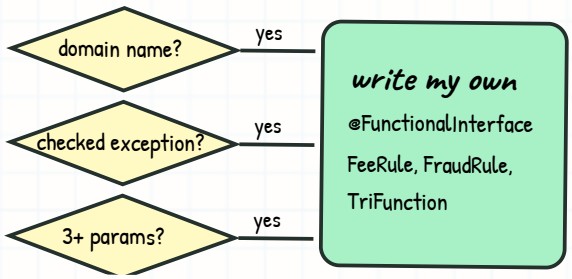
plain int ✓

`Function<Flight,Integer>` puts every int in a box.

`ToIntFunction<Flight>` + `mapToInt` : no box.

Booked seats = 1808, nearly full = [TR808, SQ21].

6 · Custom or built-in?



all "no" → reuse Function, Predicate…

★ Remember

- never overload `run(Supplier)` + `run(Callable)` → ambiguous
- Supplier vs Callable: only Callable throws
- old SAMs work: Runnable, Comparator, FileFilter
- role noun + verb: FeeRule.feeFor
- next → L03 lambdas in depth